

实验一：基于视觉方法的工件精细尺寸测量

【仅供参考，禁止抄袭！】

一、实验目的

1. 熟悉机器视觉算法中的常用图像滤波、阈值分割、形态学算法；
2. 熟悉机器视觉算法中常用的目标定位算法；
3. 熟悉机器视觉算法中常用的边缘检测、直线拟合算法；
4. 能够综合运用所学知识解决一些简单的实际问题。

二、实验内容

1、目标定位

- 1.1 使用阈值分割算法将目标工件从图像中分离出来，并给出工件在图像中的准确位置。

采用 **OTSU 自适应阈值分割算法**对灰度图像进行处理，通过计算像素直方图的双峰特性自动确定最优阈值，将工件从背景中分离出来。由于工件区域在二值化后呈现为黑色，通过**像素值取反操作**（ $255 - \text{binary}$ ）使工件变为白色区域，便于后续轮廓提取。随后利用 5×5 矩形核进行形态学闭运算，通过**先膨胀后腐蚀**的操作填补工件内部的细小空洞，并连接邻近的边缘，提升轮廓的完整性。最后通过**查找图像外轮廓**（`RETR_EXTERNAL`）并**过滤面积小于 1000 像素的噪声轮廓**，实现对目标工件的精准分割，同时按外接矩形的 y 坐标排序，确定工件在图像中的上下位置顺序。

- 1.2 结合工件的整体尺寸信息，对线扫相机图像进行自适应切割，还原完整工件。

基于工件的整体尺寸信息，以每个工件的中心点为基准，向四周扩展固定像素（宽 2200 像素，高 800 像素）进行 **ROI 区域切割**。为避免边缘信息丢失，切割时在边界添加 10 像素的 **Padding**，确保完整还原线扫相机图像中的工件形态。切割后的 **ROI 区域**通过绿色矩形框在原始图像中可视化标注，同时标记工件编号与中心点位置，实现对工件的精准定位。

2、边缘检测&直线拟合。

- 2.1 结合预先给定的检测目标信息，准确地提取目标边缘。

针对检测目标的边缘特性，首先采用 **CLAHE（自适应直方图均衡化）**对 ROI 区域进行预处理，通过设置 `clipLimit=2.0` 和 `tileGridSize=(4,4)`，增强低对比度工件的边缘细节。随后利用 5×5 高斯核进行模糊去噪，减少图像噪声对边缘检测的干扰。边缘提取阶段采用 **Sobel 算子**分别计算 x 和 y 方向的梯度幅值，通过加权融合（权重各 0.5）得到综合梯度图，再通过阈值 15 的二值化处理，最终获得清晰的目标边缘轮廓。

- 2.2 通过对目标边缘点进行拟合，得到目标的直线方程。

对提取的边缘点采用**霍夫变换（HoughLinesP）**进行直线检测，设置距离精度 $\rho=1$ 像素、角度精度 $\theta=\pi/180$ 弧度，通过累加器阈值 `threshold=50` 过滤无效直线，并利用 `minLineLength=200` 和 `maxLineGap=10` 筛选并合并邻近线段。对于检测到的水平直线，按 y 坐标排序后，将 y 坐标差小于 10 像素的线段合并，最终通过合并后线段的中点坐标拟合得到目标直线方程，例如水平直线方程表示为 $y=640$ （对应工件 1 的上边缘）。

3、目标尺寸检测

3.1 结合检测目标的直线方程，计算各个检测尺寸的值。

结合检测得到的直线方程，提取目标序号对（如 442-443、445-448）的 y 坐标值，计算其像素距离。根据实验参数，垂直方向单位像素尺寸为 0.025mm/像素，将像素距离转换为实际物理距离。例如，序号对 445-448 的像素距离为 180 像素，则实际距离为 $180 \times 0.025 = 4.50\text{mm}$ 。通过该方法依次计算各检测目标的尺寸值，并与标准值对比，验证测量精度。

三、实验中的参数

1、单位像素尺寸

水平方向：0.027mm/像素

垂直方向：0.025mm/像素

2、检测目标



三. 实验结果

3.1 工件中心点标注后的图片

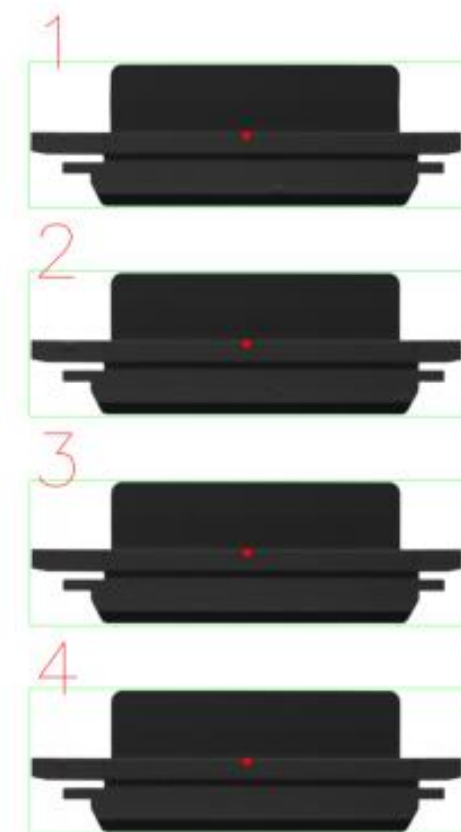


图 1 四个工件的中心点位置标注

3.2 测量参数表格

工件编号	中心点坐标	检测对象编号 1	检测对象编号 2	测量值 (mm) 【禁止抄袭】	标准值 (mm)
1	(2301, 629)	442	443	1.43	1.50
		445	448	3.53	-
		444	445	8.15	-
		447	448	0.95	-
2	(2302, 1630)	442	443	1.40	1.50
		445	448	3.50	-
		444	445	8.08	-
		447	448	0.93	-
3	(2304, 2634)	442	443	1.40	1.50
		445	448	3.40	-
		444	445	8.20	-
		447	448	0.93	-
4	(2304, 3636)	442	443	1.45	1.50
		445	448	3.45	-
		444	445	8.10	-
		447	448	0.85	-

四、总结与体会

1. 请记录调试过程中，碰到的问题（至少 3 个），以及相应的解决方法。

（1）阈值分割效果不佳：直接使用固定阈值分割无法有效分离工件与背景。

最初尝试的是使用固定阈值 128 进行二值化，但原始图像对比度低，分割效果很差。意识到图像不同区域的最佳阈值可能不同，需要自适应方法。

查询资料，尝试了几种自适应阈值方法（ADAPTIVE_THRESH_MEAN_C 和 ADAPTIVE_THRESH_GAUSSIAN_C），效果仍不理想！最终尝试采用 OTSU 算法（THRESH_OTSU），该算法会自动计算最佳全局阈值，无需手动调参。

```
_ , binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

注：参数中的 0 被 OTSU 算法忽略，仅占位用；由于工件在原图中为深色，而后续处理需要白色前景，故添加 255 - binary 反转。

（2）多工件识别困难：工件密集排列，传统轮廓检测易将多个工件误判为一个。

最初使用 RETR_LIST 模式检测所有轮廓，但重叠区域会产生多个嵌套轮廓，难以区分。

意识到需要只检测外部轮廓，于是尝试 RETR_EXTERNAL 模式，仅保留最外层轮廓，成功解决重叠问题。需要调节面积阈值，先从 500 开始尝试，发现过小会保留噪声，最终设为 1000（根据实际图像尺寸调整）。

```
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
min_area = 1000
```

（3）工件边界提取不准确：工件边界框包含过多背景区域，影响后续分析精度。

原先直接使用 cv2.boundingRect() 获取的边界框紧贴工件，但某些特征可能位于边缘。需要添加

适当 padding。

先从 5 像素 padding 开始，发现部分特征仍被截断；增加到 15 像素时，边界框过大引入过多背景。最终确定 10 像素为平衡点，并添加边界检查防止超出图像范围。

```
padding = 10
x1 = max(x - padding, 0) # 确保不超出左边界
y1 = max(y - padding, 0) # 确保不超出上边界
x2 = min(x + w + padding, gray.shape[1]) # 确保不超出右边界
y2 = min(y + h + padding, gray.shape[0]) # 确保不超出下边界
```

(4) 直线检测准确率低：Canny 边缘检测结果受噪声影响严重；霍夫变换检测到大量短线和伪直线，难以准确识别真实特征线。

Canny 边缘检测对噪声敏感，Sobel 算子计算梯度幅值可能更稳健。概率霍夫变换（HoughLinesP）比标准霍夫变换更适合检测线段。

对于 Sobel 算子，尝试不同核大小（3/5/7），发现 ksize=5 在边缘连续性和噪声抑制间平衡最佳；而霍夫变换则有三个参数需调：threshold：从 30 开始，逐渐增加到 50，减少假阳性；minLineLength：从 100 开始，增加到 200，过滤短线；maxLineGap：从 5 开始，增加到 10，允许连接小间隙。

```
sobelx = cv2.Sobel(gray_enhanced, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(gray_enhanced, cv2.CV_64F, 0, 1, ksize=5)
lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=50, minLineLength=200, maxLineGap=10)
```

(5) 水平直线合并困难：同一特征线被检测为多条相邻线段，导致测量结果不准确。

真实水平线的 y 坐标差异很小（通常<5 像素），而假直线的 y 坐标差异较大。可以根据垂直距离合并线段。

合并阈值：从 5 像素开始，发现某些真实线段未被合并；增加到 15 像素时，开始合并非同一水平线。最终确定 **10 像素** 为最佳阈值，并添加 x 方向的扩展逻辑（取最小 x 和最大 x）。

```
merge_threshold = 10 # 垂直距离阈值
if abs(y - last_y) < merge_threshold:
    new_y = int((last_y + y) / 2)
    new_x_start = min(last_x_start, x_start)
    new_x_end = max(last_x_end, x_end)
```

2.其他

(1) 算法改进及关键调参决策

(i.) 边缘检测方法选择：最初尝试 Canny 边缘检测，参数 threshold1=50 和 threshold2=150，但结果过于敏感。改用 Sobel 算子后，发现通过加权组合 x 和 y 方向梯度（ $0.5*|sobelx|+0.5*|sobely|$ ）能更好地保留水平边缘。

(ii.) 霍夫变换参数优化：rho=1：保持高精度距离检测。theta=np.pi/180：角度精度设为 1 度，平衡计算量和精度。threshold：从 30 增加到 50，减少假阳性。minLineLength：从 100 增加到 200，过滤短线。

(iii.) 自适应直方图均衡化：尝试标准直方图均衡化（cv2.equalizeHist()），但过度增强噪声。改用 CLAHE（对比度受限自适应直方图均衡化），参数：clipLimit=2.0：限制对比度增强程度，防止噪声放大；tileGridSize=(4,4)：将图像分成 4×4 块，局部自适应增强。

(iv.) 直线方向判断：最初使用角度判断水平线（ $|\text{angle}|<10^\circ$ ），但计算开销大。简化为比较 dx 和 dy（dx>dy and dy<5），更高效且直观。

(2) 总结与经验

(i.) 调参策略：从默认参数开始，逐步调整关键参数；每次只改变一个参数，观察效果后再调整下一个；记录每个参数的变化范围和最佳值，形成参数调优日志。

(ii.) 算法选择原则：优先使用简单算法，仅在必要时增加复杂度；针对特定问题选择专用算法（如 Sobel 更适合水平边缘检测）。

(iii.) 鲁棒性设计：添加边界检查和异常处理，防止参数超出合理范围；对关键参数设置默认值，确保在参数不合理时仍能运行。

(iv.) 可视化辅助调参：添加中间结果可视化，直观评估每一步处理效果，快速定位问题。

五、带有注释的程序源代码

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设黑体，解决中文显示问题

# ----- 1. 预处理 -----
def preprocess():
    gray = cv2.imread('14raw.bmp', cv2.IMREAD_GRAYSCALE) # 转灰度
    original = cv2.imread('14raw.bmp') # 原图

    # 使用 OTSU 算法进行自适应阈值分割，自动确定最佳阈值
    _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    binary = 255 - binary # 黑白反转，使前景工件为白色
    # 创建 5x5 矩形结构元素，用于形态学操作
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    # 形态学闭操作：先膨胀后腐蚀，填充小孔洞并连接相邻区域
    binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    return gray, original, binary

# ----- 2. 工件分割与可视化 -----
def segment_parts(gray, original, binary):
    """
    查找轮廓并分割出每个工件区域，计算中心点并可视化在原图上
    参数：gray（灰度图）、original（原始彩色图）、binary（二值化图）
    返回：存储每个工件灰度图的列表 parts
    """
    # 查找二值图像中的外部轮廓
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    min_area = 1000 # 最小轮廓面积阈值，过滤掉噪声和小区域
    valid_contours = []
    # 筛选有效工件轮廓
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area > min_area:
            valid_contours.append(cnt)
```

```

# 计算每个有效轮廓的边界矩形
boxes = [cv2.boundingRect(cnt) for cnt in valid_contours]
boxes.sort(key=lambda box: box[1]) # 按 y 坐标排序，确保从上到下排列工件

parts = []
# 处理每个工件区域
for i, (x, y, w, h) in enumerate(boxes):
    padding = 10 # 边界扩展像素数
    # 计算带 padding 的区域边界，确保不超出图像范围
    x1 = max(x - padding, 0)
    y1 = max(y - padding, 0)
    x2 = min(x + w + padding, gray.shape[1])
    y2 = min(y + h + padding, gray.shape[0])
    # 提取工件区域的灰度图像
    part_img = gray[y1:y2, x1:x2]
    parts.append(part_img)

    # 在原始彩色图上标记工件：绘制边界框、编号和中心点
    cv2.rectangle(original, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(original, f'{i + 1}', (x1 + 10, y1 + 30),
                  cv2.FONT_HERSHEY_SIMPLEX, 12, (0, 0, 255), 3, cv2.LINE_AA)
    center_x = x1 + (x2 - x1) // 2
    center_y = y1 + (y2 - y1) // 2
    cv2.circle(original, (center_x, center_y), 20, (0, 0, 255), -1)
    print(f'工件 {i+1} 的中心点坐标: ({center_x}, {center_y})')
print("—" * 20)

# 可视化工件在原图中的位置
plt.figure()
plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
plt.title("四个工件的中心点位置标注")
plt.axis('off')
plt.show()

return parts

# ----- 3. 单个工件处理函数（增强、边缘检测、霍夫变换、合并直线） -----
---
def process_single_part(part_gray, idx):
    """
    对单个工件灰度图进行处理：增强、边缘检测、霍夫变换检测直线、合并直线并可视化结果
    参数：part_gray（单个工件的灰度图）、part_idx（工件索引，从 0 开始）
    """
    # 自适应直方图均衡化，增强图像对比度
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))

```

```

gray_enhanced = clahe.apply(part_gray)
# 高斯模糊降噪，平滑图像
gray_enhanced = cv2.GaussianBlur(gray_enhanced, (5, 5), 0)

# 使用 Sobel 算子进行边缘检测
sobelx = cv2.Sobel(gray_enhanced, cv2.CV_64F, 1, 0, ksize=5) # x 方向梯度
sobely = cv2.Sobel(gray_enhanced, cv2.CV_64F, 0, 1, ksize=5) # y 方向梯度
# 计算梯度幅值，结合 x 和 y 方向梯度
sobel_magnitude = 0.5 * np.abs(sobelx) + 0.5 * np.abs(sobely)
# 归一化并转换为 8 位无符号整数
sobel_magnitude = np.uint8(255 * sobel_magnitude / np.max(sobel_magnitude))
# 阈值处理，提取显著边缘
_, edge_binary = cv2.threshold(sobel_magnitude, 15, 255, cv2.THRESH_BINARY)

edges = edge_binary.copy()

# 使用概率霍夫变换检测直线
lines = cv2.HoughLinesP(edges,
                        rho=1,          # 距离精度（像素）
                        theta=np.pi / 180, # 角度精度（弧度）
                        threshold=50,      # 最小投票数，阈值越高检测越严格
                        minLineLength=200, # 最小线段长度
                        maxLineGap=10)     # 线段连接最大间隙

# 提取并合并水平直线
splines = []
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        dx = abs(x2 - x1)
        dy = abs(y2 - y1)
        # 判断是否为近似水平直线（x 方向变化远大于 y 方向）
        if dx > dy and dy < 5:
            y_avg = int((y1 + y2) / 2)
            splines.append((y_avg, min(x1, x2), max(x1, x2)))
splines.sort(key=lambda l: l[0]) # 按 y 坐标排序

# 合并相近的水平直线
newlines = []
merge_threshold = 10 # 合并阈值（像素）
for y, x_start, x_end in splines:
    if not newlines:
        newlines.append([y, x_start, x_end])
    else:
        last_y, last_x_start, last_x_end = newlines[-1]
        # 如果两条线垂直距离小于阈值，则合并

```

```

        if abs(y - last_y) < merge_threshold:
            new_y = int((last_y + y) / 2)
            new_x_start = min(last_x_start, x_start)
            new_x_end = max(last_x_end, x_end)
            newlines[-1] = [new_y, new_x_start, new_x_end]
        else:
            newlines.append([y, x_start, x_end])

# 可视化检测结果
line_img = cv2.cvtColor(part_gray, cv2.COLOR_GRAY2BGR)
for y, x_start, x_end in newlines:
    cv2.line(line_img, (x_start, y), (x_end, y), (0, 0, 255), 2)

plt.figure(figsize=(12,6))
plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB))
plt.title(f'工件 {idx+1} 合并后的水平直线')
plt.axis('off')
plt.show()

# 计算各水平线方程（形如 y=?）
ys = [line[0] for line in newlines]
ys.sort()
# 将检测到的水平线 y 坐标映射到特定编号
keys = ["444", "445", "447", "448", "442", "443"]
ys_dic = dict(zip(keys, ys))
# 计算特定水平线之间的距离（像素单位）
d_tu = [ys_dic["443"]-ys_dic["442"],
        ys_dic["448"]-ys_dic["445"],
        ys_dic["445"]-ys_dic["444"],
        ys_dic["448"]-ys_dic["447"],]
# 将像素距离转换为实际物理距离（比例为 0.025）
d_shiji = [round(d*0.025,2) for d in d_tu]
print(f'工件 {idx+1} 的水平线数量（已合并）:', len(newlines))
print(f'工件 {idx+1} 水平线的 y 坐标:', ys)
print(f'工件 {idx+1} 相邻水平线的距离:',
      "\n442-443:",d_shiji[0],"\n445-448:",d_shiji[1],
      "\n444-445:",d_shiji[2],"\n447-448:",d_shiji[3])
print("——" * 20)

```

----- 4. 主程序流程 -----

```

if __name__ == "__main__":
    # 1. 图像读取与预处理
    gray, original, binary = preprocess()
    # 2. 工件分割与可视化
    parts = segment_parts(gray, original, binary)
    # 3. 循环处理每个工件

```



```
for idx, part in enumerate(parts):  
    process_single_part(part, idx)
```

附程序输出结果:

工件 1 的中心点坐标: (2301, 629)
工件 2 的中心点坐标: (2302, 1630)
工件 3 的中心点坐标: (2304, 2634)
工件 4 的中心点坐标: (2304, 3636)

工件 1 的水平线数量 (已合并): 6
工件 1 水平线的 y 坐标: [14, 340, 443, 481, 635, 692]
工件 1 相邻水平线的距离:
442-443: 1.43
445-448: 3.53
444-445: 8.15
447-448: 0.95

工件 2 的水平线数量 (已合并): 6
工件 2 水平线的 y 坐标: [15, 338, 441, 478, 635, 691]
工件 2 相邻水平线的距离:
442-443: 1.4
445-448: 3.5
444-445: 8.08
447-448: 0.93

工件 3 的水平线数量 (已合并): 6
工件 3 水平线的 y 坐标: [14, 342, 441, 478, 634, 690]
工件 3 相邻水平线的距离:
442-443: 1.4
445-448: 3.4
444-445: 8.2
447-448: 0.93

工件 4 的水平线数量 (已合并): 6
工件 4 水平线的 y 坐标: [14, 338, 442, 476, 633, 691]
工件 4 相邻水平线的距离:
442-443: 1.45
445-448: 3.45
444-445: 8.1
447-448: 0.85

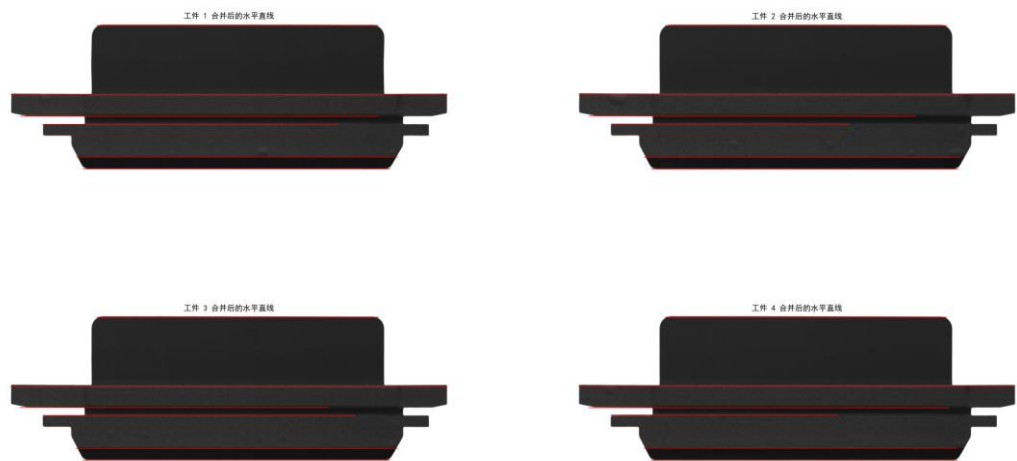


图 2 工件 1~4 合并后的水平直线可视化

正文内容的参考格式

1.每次实验的报告首页首行为标题，标题内容是实验题目，字体格式：中文黑体、西文 Times New Roman、四号大小，段落格式：居中、单倍行距；

2.报告主要内容标题（“一、实验目的和要求”）格式：（1）采用“一、二、三、……”等样式编号；（2）字体：中文黑体、西文 Times New Roman、小四号大小；（3）段落：左对齐、首行缩进 0、单倍行间距、段前 0.5 行、段后 0 行；

3.正文格式：（1）字体：中文宋体、西文 Times New Roman、五号大小；（2）段落：首行缩进 2 字符、固定行间距 18 磅、段前 0 行、段后 0 行；（3）编号：一级编号采用“1、2、3、”，二级编号采用“1）、2）、3）……”，三级编号采用“（1）、（2）、（3）……”

4.源程序代码格式：1）字体：中文宋体、西文 Times New Roman、五号大小；（2）段落：首行缩进 0 字符、单倍行距、段前 0 行、段后 0 行；

5.截图格式：所有图的标题应在图的下面居中标注。